



SVEUČILIŠTE U ZAGREBU
Fakultet elektrotehnike i računarstva

PARALELNO PROGRAMIRANJE

Poglavlje 01 - Uvod



1.1 ZAŠTO PARALELNO RAČUNANJE?

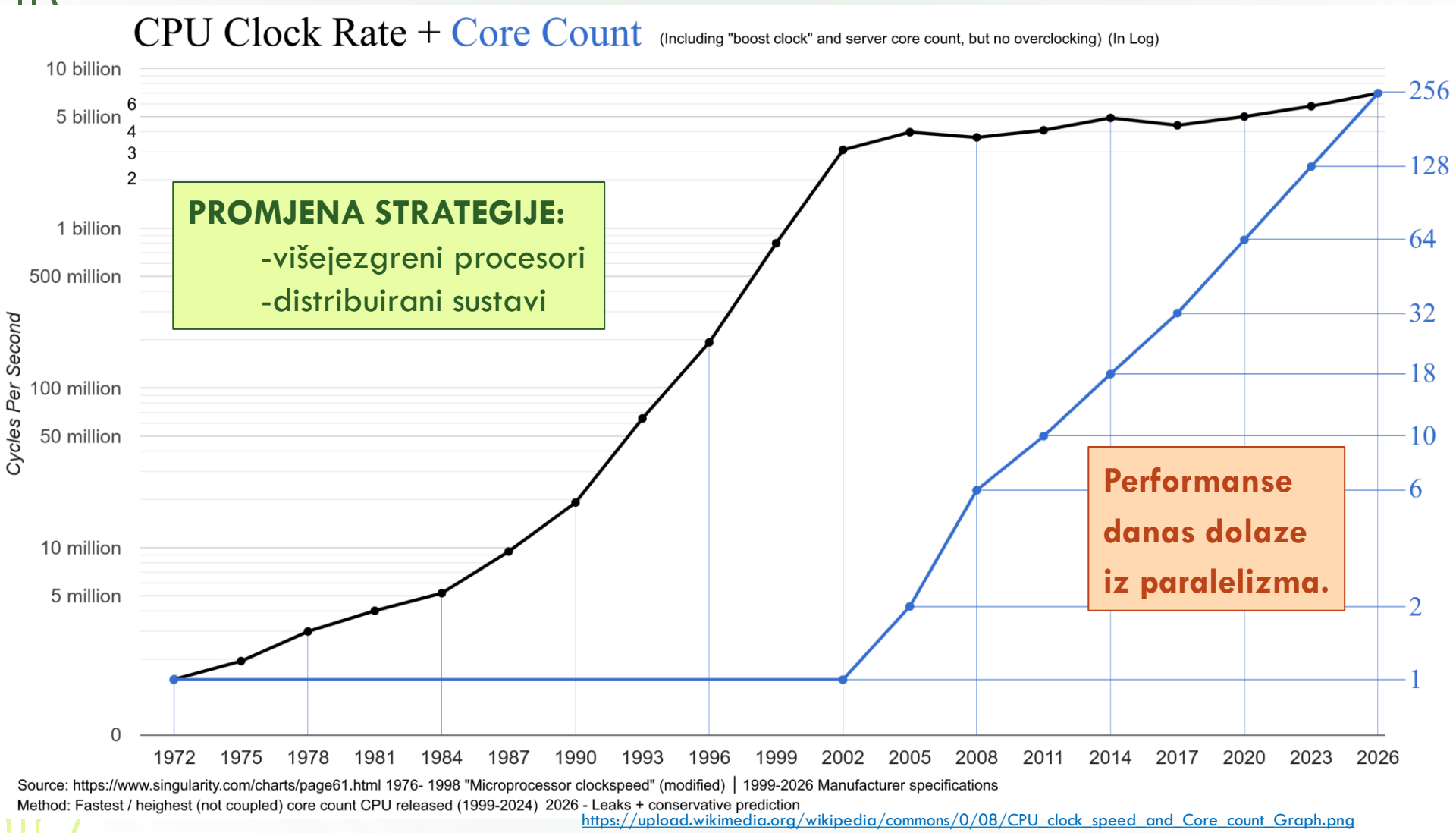
- Riješiti problem **brže** nego što bi bilo moguće slijednim izvođenjem
- Obraditi **više podataka u istom vremenu** (veća propusnost sustava)
- Omogućiti **interaktivne** i **real-time** sustave
- Iskoristiti moderni hardver koji je već paralelan
  - cjevovodi (pipelining)
  - Hyperthreading
  - nCore procesori
  - GP/GPU (General-Purpose computation na GPU-ovima)

PRIMJERI

- **Modeliranje i simulacije**
  - klimatski modeli
  - seizmičke simulacije
  - simulacije fluida i turbulencija
  - dizajn zrakoplova i automobila
  - farmacija i medicina
  - simulacije elektroničkih sustava
- **Obrada velikih količina podataka**
  - računalni vid
  - obrada videa i multimedije
  - pretraživanje i velike baze podataka
  - data mining
  - duboko učenje
  - Bioinformatika
- **Ostale primjene**
  - dekodiranje DNA
  - financijske simulacije
  - optimizacijski problemi
  - računalne igre i fizikalni enginei

ZAŠTO SU PARALELNA RAČUNALA POSTALA NUŽNA

- Nekada se performanse povećavale jednostavno:
  povećavanjem frekvencije procesora
- To više ne funkcionira:
  - **Fizikalna ograničenja**
    - zagrijavanje procesora
    - potrošnja energije
    - kašnjenje signala
  - **Frekvencije su prestale značajno rasti**
    - CPU radi na ~3–5 GHz već gotovo dva desetljeća



Source: https://www.singularity.com/charts/page61.html 1976- 1998 "Microprocessor clockspeed" (modified) | 1999-2026 Manufacturer specifications
Method: Fastest / heighest (not coupled) core count CPU released (1999-2024) 2026 - Leaks + conservative prediction
https://upload.wikimedia.org/wikipedia/commons/0/08/CPU_clock_speed_and_Core_count_Graph.png

Spec	AMD Ryzen 9 9955HX3D	AMD Threadripper PRO 9995WX	Intel Xeon 6960P
Segment	High-end laptop	Workstation	Datacenter / server
Architecture	Zen 5	Zen 5	Granite Rapids
Year	~2025	2025	2024
Cores	16	96	72
Threads	32	192	144
Base clock	2.5 GHz	2.5 GHz	2.7 GHz
Boost clock	~5.4 GHz	~5.4 GHz	~3.9 GHz
L3 cache	~128 MB (3D V-Cache)	384 MB	~432 MB
TDP	~55–75 W	~350 W	~500 W class
Process node	~4 nm class	~4 nm	~5 nm

GDJE SE PARALELIZAM NALAZI DANAS

- Unutar procesora
  - Pipelining
  - SIMD vektorske instrukcije
- Više jezgri u jednom CPU-u
  - multicore procesori
  - hyperthreading
- Akceleratori
  - GPU
  - specijalizirani AI akceleratori (FPGA...)
- Više računala
  - cluster
  - cloud
  - superračunala

ŠTO JE POTREBNO ZA PARALELNO RAČUNANJE

- Da bi paralelni sustav radio, potrebno je:
  - hardver s više procesnih jedinica
  - komunikacija između njih
  - programski model (MPI, OpenMP, CUDA, ...)
  - paralelni algoritam
- **I najvažnije:**
  - **programer koji razumije paralelizam ☺**

PRIMJER

- Potrebno je skuhati večeru od 3 slijeda (hladno predjelo, glavno jelo i desert) za 100 ljudi
- Koliko će trajati priprema?
  - Ako radi 1 osoba (pod pretpostavkom da zna kuhati ☺):
    - Jedno po jedno jelo … cca 200 sati – nemoguće!
    - Više jela istovremeno…?

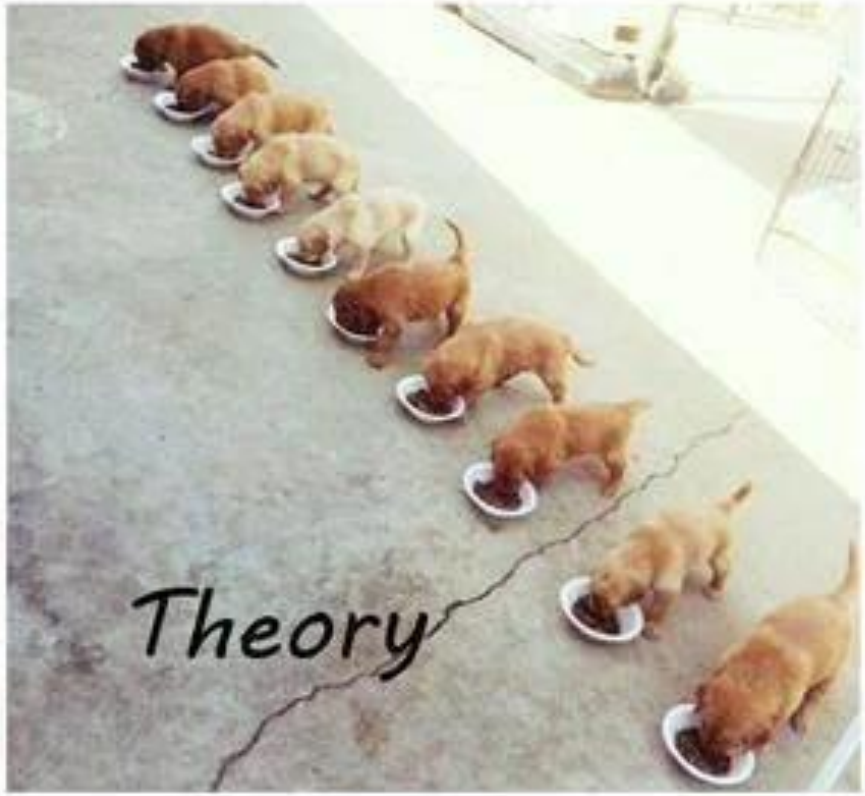
PRIMJER

- ...Koliko će trajati priprema?
  - Ako ta osoba angažira još 9 prijatelja koji znaju kuhati?
  - Razmotriti
    - Koliko je štednjaka na raspolaganju?
    - Koliko traje priprema koje komponente?
    - Trebaju li sva jela izaći u isto vrijeme?

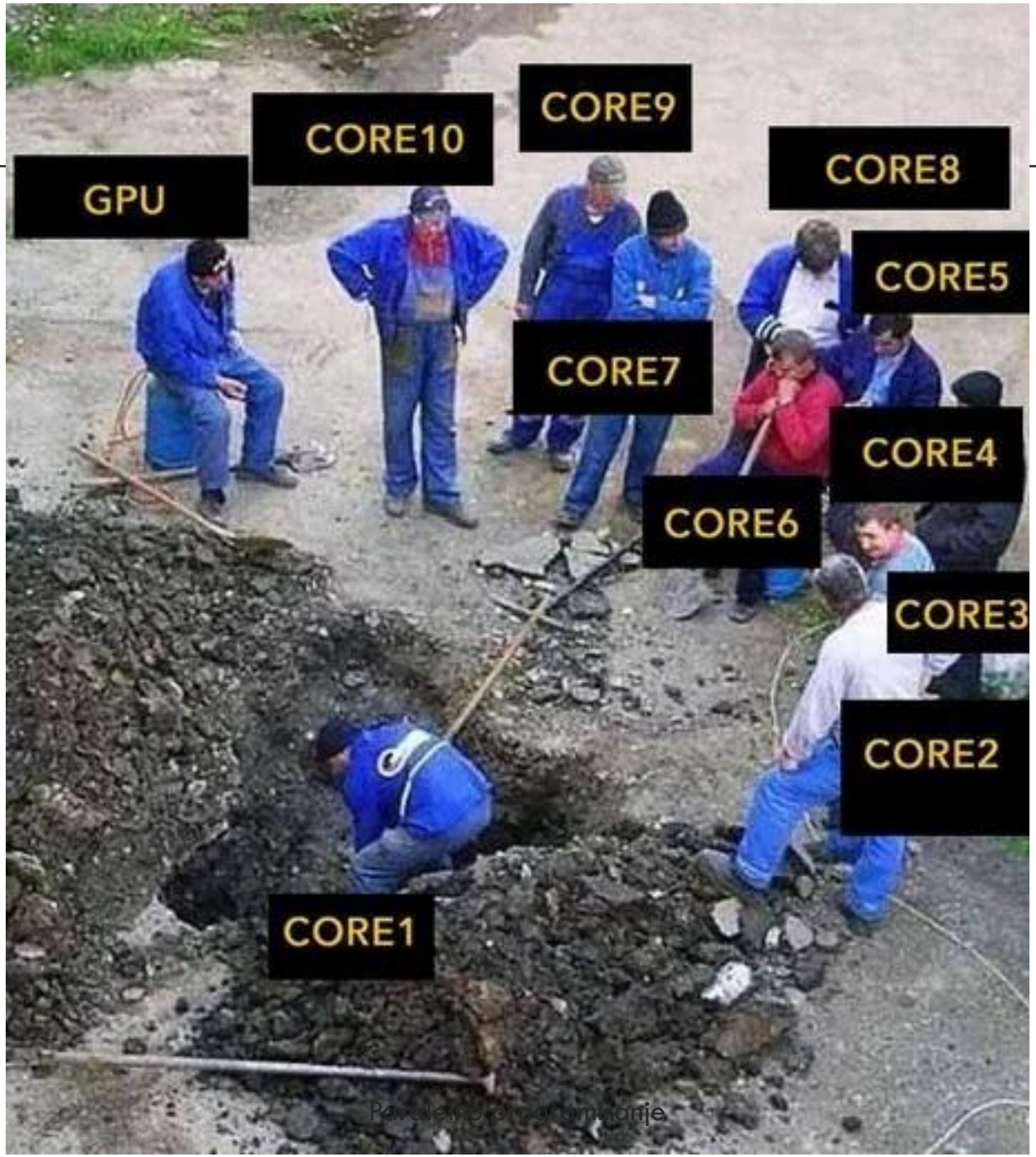
PRIMJER

- ...Što smo naučili
  - osobe koje kuhaju ~ CPU jezgre
  - recept (priprema večere) ~ algoritam
  - komponente jela (prilog, meso, salata...) ~ zadaci
  - štednjak ~ dijeljeni resurs
  - vrijeme izlaska jela ~ sinkronizacija

Multithreaded programming






GPU
CORE10
CORE9
CORE8
CORE5
CORE7
CORE4
CORE6
CORE3
CORE2
CORE1

1.2 MODELI PARALELNIH RAČUNALA

- Zašto je potreban model?
  - ➔omogućavanje razvoja algoritama koji su primjenjivi na velikom broju različitih računala (a ne samo na određenom računalu)

- Model bi trebao biti:
  - **jednostavan** - učinkovito programiranje
  - **realan** - algoritmi napisani za model se lako primjene na stvarna računala
- **Paralelno računalo:** skup procesora koji mogu zajednički rješavati neki računalni problem

FLYNNOVA TAKSONOMIJA

- **SISD (Single Instruction, Single Data Stream)**
  - Von Neumannov model računala (1 processor + 1 memorija), jedna instrukcija obrađuje jedan (skalarni) podatak
- **SIMD (Single Instruction, Multiple Data Stream)**
  - jedna instrukcija obrađuje **više podataka istodobno** (npr. *vektori, matrice…*)
  - polje procesora koji izvode **jednake instrukcije** na **različitim podacima** rade **sinkronizirano** (*lock-step* način)
  - **nedostatak: grananja** unutar petlji moraju se primijeniti na sve procesore
  - primjene ograničene na probleme jednolike obrade velikog skupa podataka (obrada slike, numeričke simulacije…)
  - moderni procesori uključuju **posebne instrukcije za vektorske operande**
  - danas: najbliža usporedba s **GPU procesorima**
- **MISD (Multiple Instruction, Single Data Stream)**
- **MIMD (Multiple Instruction, Multiple Data Stream)**
  - više procesora izvodi **različite instrukcije** na **različitim podacima**
  - prednosti: paralelno izvođenje različitih poslova, neovisnost procesora
  - nedostaci: ujednačavanje opterećenja (*load-balancing*), teže za programirati

PODJELA MODELA PO MEMORIJSKOJ STRUKTURI

- Zajednička memorija (*shared memory*)
  - procesori rade neovisno jedan o drugome, ali koriste **isti memorijski spremnik**
  - čitanje i pisanje je **ekskluzivno** (jedinstvena sabirnica)
  - lakše programiranje - nije potrebno dijeliti podatke među zadacima
  - nedostaci:
    - povećanje broja procesora uz jednaku količinu memorije može uzrokovati **zagušenje** (ograničeni bandwidth)
    - korisnik odgovoran za **sinkronizaciju**
- Raspodijeljena memorija (*distributed memory*)
  - više neovisnih procesora s **vlastitim spremnicima**
  - podjela podataka i sinkronizacija odvija se **porukama** (message passing)
  - nedostaci:
    - teško je postojeće podatkovne strukture prilagoditi ovom modelu
    - korisnik (programer) odgovoran za dijeljenje podataka
- NUMA model (*non uniform memory access*)
  - skup procesora (ili računala) s više memorijskih jedinica, ali **zajedničkim adresnim prostorom**
  - svaki procesor ima "vlastitu" memoriju, ali može pristupiti memoriji bilo kojeg procesora
  - **različito trajanje pristupa** različitim memorijskim lokacijama
  - npr. današnji višejezgreni procesori s višerazinskom priručnom memorijom (cache)

1.3 MODELI PARALELNIH PROGRAMA

1. **Razmjena poruka** (*message passing*)
   - više zadataka izvodi se neovisno; podaci se razmjenjuju porukama
   - **SPMD** (*single program, multiple data*) - isti program se izvodi na više procesora
   - unutar jedinstvenog programa implementiraju se različite **uloge** u sustavu (voditelj-radnik, *master-worker*)
2. **Podatkovni paralelizam** (*data parallelism*)
   - primjena **iste operacije na više elemenata** podatkovne strukture
   - potrebni prilagođeni programski jezici
3. **Zajednička memorija**
   - svi zadaci dijele isti memorijski spremnik - čitanje i pisanje je **asinkrono**
   - višeprocesni ili višedretveni model – nužna eksplicitna zaštita memorije (npr. *mutex*)
4. **Sustav zadataka i kanala** (usmjereni graf)
   - čvorovi → zadaci (mogu aw izvoditi paralelno i neovisno jedan o drugome)
   - veze → kanali kojima zadaci komuniciraju
   - poopćenje modela komunikacije porukama

1.4 SVOJSTVA PARALELNIH ALGORITAMA

- **Istodobnost (*concurrency*)**
  - mogućnost izvođenja više radnji istovremeno
- **Skalabilnost (*scalability*)**
  - mogućnost prilagođavanja **proizvoljnom broju fizičkih procesora** (*algoritam koji radi samo na x procesora je loš algoritam*)
- **Lokalnost (*locality*)**
  - veći omjer lokalnog u odnosu na udaljeni pristup memoriji – korištenje lokalne ili priručne memorije (*cache*)
- **Modularnost (*modularity*)**
  - mogućnost uporabe dijelova algoritma unutar različitih paralelnih programa

UJEDNAČAVANJE OPTEREĆENJA I ZRNATOST

- Ujednačavanje opterećenja (*load balancing*)
  - raspodjela zadataka u smislu **vremenske učinkovitosti algoritma**
  - neujednačeno izvođenje: svi zadaci čekaju da jedan završi posao
  - poseban **problem u raznorodnim** (*heterogeneus*) računalnim sustavima
- Zrnatost (granularity)
  - omjer između **količine računanja** (lokalnog rada) i **količine komunikacije** (nelokalnog rada)
  - Sitnozrnatost (*fine-grained*)
    - **mala količina računanja** između uzastopnih udaljenih komunikacija
    - lakše ujednačavanje opterećenja, ali **veći trošak komunikacije** (*overhead*)
  - Krupnozrnatost (*coarse-grained*)
    - **velika količina računanja** u odnosu na komunikaciju
    - više mogućnosti ubrzavanja ali **teža kontrola ujednačavanja opterećenja**

AMDAHLOV ZAKON

- jedan od načina opisivanja učinkovitosti paralelnog algoritma
- pretpostavke:
  - veličina problema (količina posla) je konstantna
  - udio programa koji se može paralelizirati (p) neovisan o broju procesora

$$ubrzanje = \frac{1}{s + \frac{p}{N_P}}$$

Paralelni dio posla *(s+p=1)*

Serijski dio posla *(koji se ne može paralelizirati)*

Broj procesora *(koji izvode paralelni posao)*

1.5 PRETVORBA SLIJEDNOG U PARALELNI ALGORITAM

- Funkcionalna dekompozicija
- Podatkovna dekompozicija
- Uloge (*master/worker*)?

PRIMJER

- **Računanje broja Pi (π)**
  - primjer algoritma za računanje broja *Pi*: računanje omjera površine kruga upisanog kvadratu:

$$\pi = 4 \cdot \frac{P_{KRUG}}{P_{KVADRAT}}$$

  - slijedni algoritam:

```
b_tocaka = 100000; // bilo koji veliki broj – sto veci to preciznije
b_tocaka_krug = 0;

ponovi b_tocaka puta
  R1, R2 = dva slucajna broja [0,2r];
  ako (tocka (R1,R2) unutar kruga)
     b_tocaka_krug++;
kraj_ponovi

PI = 4*b_tocaka_krug/b_tocaka;
```

PRIMJER (NASTAVAK)

- Paralelizacija petlje:
  - svaki procesor izvodi svoj broj iteracija petlje – **trivijalno paraleno**
  - SPMD model (komunikacija porukama) – jedan proces **voditelj** (*master*) **sakuplja rezultate** od svih ostalih (radnici, *workers*)

```
b_tocaka = 100000; // bilo koji veliki broj – sto veci to preciznije
b_tocaka_krug = 0;
p = broj procesa;
b_tocaka_p = b_tocaka / p;

ponovi b_tocaka_p puta
  R1, R2 = dva slucajna broja [0,2r];
  ako (tocka (R1,R2) unutar kruga)
     b_tocaka_krug++;
kraj_ponovi

ako sam voditelj
  primi b_tocaka_krug od svih radnika;
  izracunaj PI (pomocu vlastite i zbroja svih dobivenih vrijednosti);
inace ako sam radnik
  posalji voditelju b_tocaka_krug;
```

PRIMJER (NASTAVAK)

- Potencijalni problem: računala na kojima izvodimo procese različitih su brzina → **proces voditelj uvijek čeka najsporijeg radnika**
  - potrebno je ujednačiti opterećenje
  - strategija: **podjela posla** na više manjih dijelova (broj dijelova >> broja procesora);

```
b_tocaka = 100000; // bilo koji veliki broj – sto veci to preciznije
b_tocaka_krug = 0;
jobs = broj poslova;
b_tocaka_p = b_tocaka / jobs;

ako sam voditelj
   ponovi dok ima poslova
        primi rezultat od radnika;   // prvi put samo kao prijava
        posalji radniku posao;
   kraj_ponovi
   posalji svim radnicima: nema poslova;
   izracunaj PI (pomocu vlastite i zbroja svih dobivenih vrijednosti);

inace ako sam radnik
   posalji voditelju prijavu;
   primi posao;
   ponovi dok ima poslova
        ponovi b_tocaka_p puta
                R1, R2 = dva slucajna broja [0,2r];
                ako (tocka (R1,R2) unutar kruga)
                        b_tocaka_krug++;
        kraj_ponovi
        posalji voditelju b_tocaka_krug;
        primi posao;
   kraj_ponovi
```